رواد مصر الرقمية

وزارة الاتصـــــالات
وتكنولوجيا المعلومات

# Mobile Application Development

## Jetpack Compose

# What’s the jetpack compose?

**Jetpack Compose** is Android's **recommended modern toolkit for building native UI**.
It simplifies and accelerates UI development on Android.
Quickly bring your app to life with less code, powerful tools, accelerate development, and intuitive Kotlin APIs.

Jetpack Compose is totally declarative programming, which means you can describe your user interface by invoking a set of composable. For the past ten years, we have been using a traditional way of imperative UI design.

The **@Composable** annotation is used to identify composable functions.
This annotation is required for composable functions because it informs the compiler that this function adds UI to the View Hierarchy.
Composable functions **can call** standard functions,
but Composable **can only be called** by other Composable functions.

```
@Composable
fun JetpackCompose() {
    Card {
        var expanded by remember { mutableStateOf(false) }
        Column(Modifier.clickable { expanded = !expanded }) {
            Image(painterResource(R.drawable.jetpack_compose))
            AnimatedVisibility(expanded) {
                Text(
                    text = "Jetpack Compose",
                    style = MaterialTheme.typography.h2,
                )
            }
        }
    }
}
```

# What is declarative programming?

**Declarative programming** is when you write your code in such a way that it describes what you want to do, and not how you want to do it. It is left up to the compiler to figure out how.

**Imperative** tells the computer how to do things,

whereas declarative focuses on what we want to get from the computer

# Declarative Programming vs. Imperative Programming

## Declarative programming

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            Text(text = "Hello World")
        }
    }
}
```

## Imperative Programming

Xml:

```
<TextView
    android:id="@+id/tv_name"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
/>
```

Java:

```
public class MainActivity extends AppCompatActivity {

    TextView tvName;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_checkbox);
        tvName = findViewById(R.id.tv_name);
        tvName.setText("Hello World");
    }
}
```

# What are the basic features of Jetpack Compose?

## Composable function

It’s the same as any function in programming.

But we need to annotate with the **@Composable** annotation.

Syntax:

```
@Composable
fun MethodName(parameter: String) {
    //your content
}
```

## Preview

Adding **@Preview()** annotation before the composable function to see the preview of our UI.

```
@Preview()
@Composable
fun DefaultPreview() {
    Text("Hello World!")
}
```

## Hello World Application

We created a new composable function - **Greeting()**.

We use the **Text()** as content.
It's an in-built composable function.

We can add composable functions
to another composable function.

After creating a composable function,
we can use it as content.

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            Greeting("World")
        }
    }
}
@Composable
fun Greeting(name: String) {
    Text(text = "Hello $name!")
}
```

## Composable function

Using Compose, you can build your user interface by defining a set of composable functions that take in data and **emit UI** elements.

A simple example is a Greeting widget, which takes in a String and emits a Text widget that displays a greeting message.


```
@Composable
fun Greeting(name: String) {
    Text("Hello $name")
}
```

**A few noteworthy things about this function:**

- The function is annotated with the **@Composable** annotation.
  All Composable functions **must** have this annotation; this annotation informs the Compose compiler that this function is intended to **convert data into UI**.

- The function takes in data.
  Composable functions can accept parameters, which allow the app logic to describe the UI.
  In this case, our widget accepts a String so it can greet the user by name.

- The function displays text in the UI.
  It does so by calling the **Text()** composable function, which actually creates the text UI element.
  Composable functions emit UI hierarchy by calling other composable functions.

- The function doesn't return anything. Compose functions that emit UI do not need to return anything, because they describe the desired screen state instead of constructing UI widgets.

**This function is fast, idempotent, and free of side effects.**

- The function behaves the same way when called multiple times with the same argument, and it does not use other values such as global variables or calls to random().

- The function describes the UI without any side-effects, such as modifying properties or global variables.

**Dynamic content**

Because composable functions are written in Kotlin instead of XML, they can be as dynamic as any other Kotlin code.

This function takes in a list of names and generates a greeting for each user.

```
@Composable
fun Greeting(names: List<String>) {
    for (name in names) {
        Text("Hello $name")
    }
}
```

## Recomposition

In an **imperative UI** model, to change a widget, you call a setter on the widget to change its internal state.
In **Compose**, you call the composable function again with new data.
Doing so causes the function to be **recomposed**, and the widgets emitted by the function are redrawn, if necessary, with new data. The Compose framework can intelligently recompose **only** the components that **changed**.

Every time the button is clicked, the caller updates the value of clicks.
Compose calls the lambda with the Text function again to show the new value; this process is called **recomposition**.
Other functions that don't depend on the value are **not** recomposed.

```
@Composable
fun ClickCounter(clicks: Int, onClick: () -> Unit) {
    Button(onClick = onClick) {
        Text("I've been clicked $clicks times")
    }
}
```

Recomposing the entire UI tree can be computationally expensive, which uses computing power and battery life.
Compose solves this problem with this intelligent **recomposition**.

**Recomposition** is the process of calling your composable functions again **when inputs change**.
When Compose recomposes based on new inputs, it only calls the functions or lambdas that might have changed, and skips the rest.
By skipping all functions or lambdas that don't have changed parameters, Compose can recompose efficiently.

Never depend on side effects from executing composable functions, since a function's recomposition may be skipped.
If you do, users may experience strange and unpredictable behavior in your app.
A **side-effect** is any change that is visible to the rest of your app. For example, these actions are all dangerous side effects:

- Writing to a property of a shared object
- Updating an observable in ViewModel
- Updating shared preferences

**Composable functions can execute in any order**

If you look at the code for a composable function, you might assume that the code is run in the order it appears.

But this isn't guaranteed to be true.

If a composable function contains calls to other composable functions, those functions might run in any order.

Compose has the option of recognizing that some UI elements are higher priority than others, and drawing them first.

The calls to StartScreen, MiddleScreen, and EndScreen might happen in any order.

```
@Composable
fun ButtonRow() {
    MyFancyNavigation {
        StartScreen()
        MiddleScreen()
        EndScreen()
    }
}
```

**Composable functions could be run in parallel**

Composable functions cannot currently be run in parallel, but you should write Compose code in a multithreaded way.

Compose could optimize recomposition by running composable functions in parallel.
This would let Compose take advantage of multiple cores and run composable functions not on the screen at a lower priority.

Compose might execute some composable functions on background threads rather than the UI thread (main thread).

Functions that are not currently rendering UI on the screen might be run at a lower priority, so system resources are better utilized.

**Composable functions could be run in parallel**

A composable function might be called from several threads at the same time. For example, if a composable interacts with a ViewModel, Compose could call ViewModel methods on different threads simultaneously.

**No side effects in composables.** To keep things thread-safe, composable functions should:

- Not modify shared state directly (e.g., don't change a variable in a ViewModel or outside of the composable).
- Not have side-effects (e.g., don't log, write to a database, or modify the UI directly).
  Instead, trigger side-effects in event-driven callbacks, like onClick, which always run on the UI thread.

Avoid modifying variables. Any variables declared inside a composable lambda should not be modified because:

- It is not thread-safe (Compose might invoke the composable on different threads).
- It introduces unintended side-effects that violate Compose's functional programming principles.

Do not use mutable variables or shared data that multiple threads might try to modify simultaneously.

```
var count = 0
@Composable
fun Counter() {
    count++  // Modifying shared state inside a composable
    Text("Count: $count")
}
```

Don’t do this

```
@Composable
fun Counter() {
    var count by remember { mutableStateOf(0) }
    Button(onClick = { count++ }) {
        Text("Count: $count")
    }
}
```

Do this

Use **LaunchedEffect**, **SideEffect**, or other side-effect APIs to manage effects explicitly.

```
@Composable
fun FetchData(viewModel: MyViewModel) {
    LaunchedEffect(Unit) {
        viewModel.fetchData() // Safe, happens once
    }
}
```

## Preview properties

**1- Dimension**

By default, the size **@Preview** is automatically chosen for the wrap of its contents.

To set the size manually, add the heightDp and the parameters widthDp.

These values are already interpreted as dp, so you don't need to add .dp

**2- Use with different devices**

You can edit the device parameter of the Preview annotation to define configurations for your composables in different devices.

**3- SystemUI**

If you need to display status and action bars in a preview, add the parameter showSystemUi

**4- Background**

By default, the composable is displayed with a transparent background.

To add a background, add the showBackground and parameters backgroundColor.

Note that this backgroundColor is an LongARGB, not a value Color.

**5-Dynamic Color**

If you have enabled dynamic color in your app, use the attribute wallpaper to change the background and see how your UI reacts to the background chosen by different users

**6- Localization**

To test different locale settings, add the parameter locale:

**7-UIMode**

The parameter uiMode can take one of the constants Configuration.UI_* and allows you to change the preview behavior accordingly. For example, you can set the preview to Night mode to see how the theme reacts.

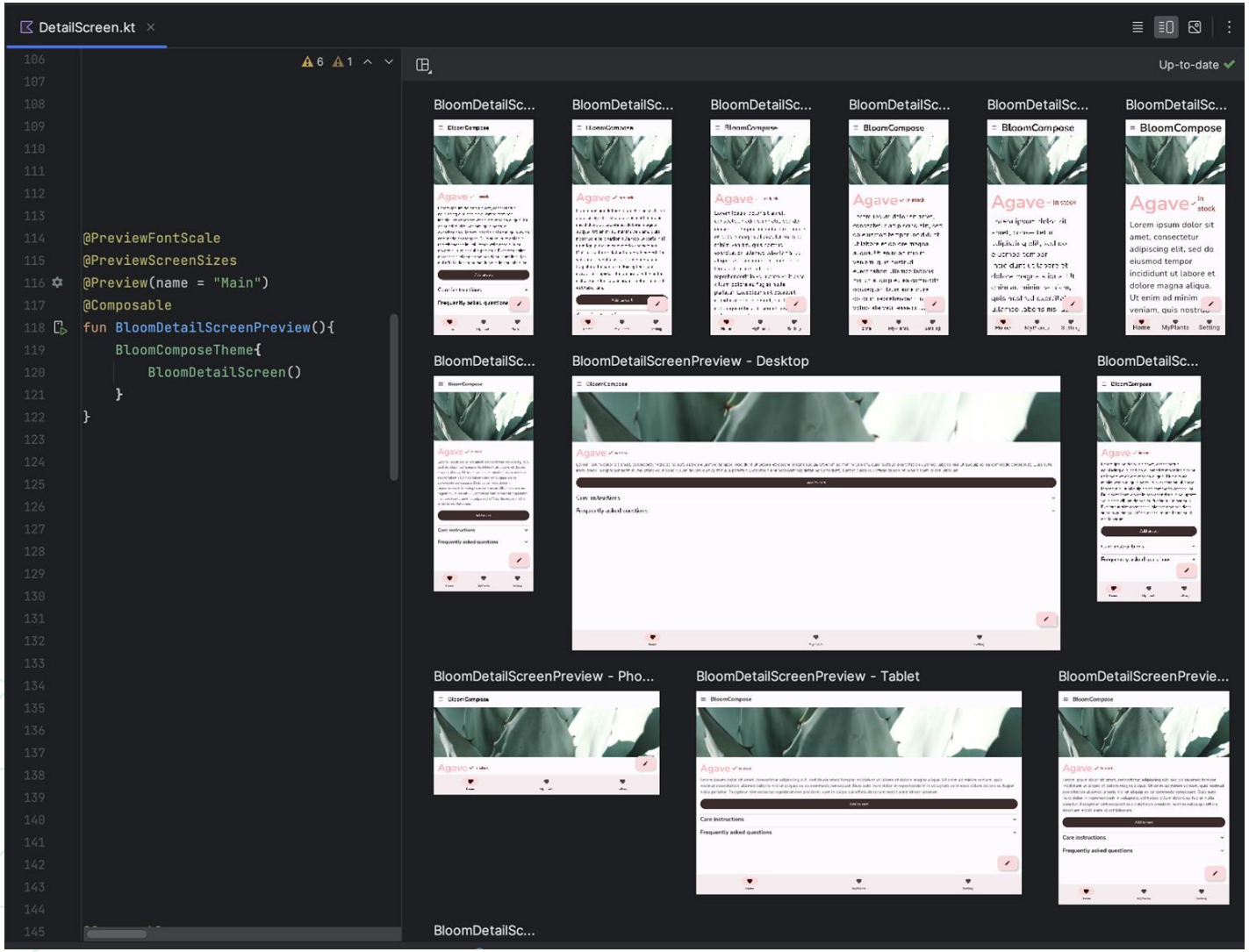
**Interactive mode**

Interactive mode lets you interact with a preview similar to how you would on a device running the program, such as a smartphone or tablet. Interactive mode is isolated in a sandbox environment (i.e., isolated from other previews), where you can click on elements and enter user input into the preview.

It's a quick way to test different states, gestures, and even animations of your composable.

## 9- Multipreview templates

androidx.compose.ui:ui-tooling-preview 1.6.0-alpha01+ introduces Multipreview API templates: @PreviewScreenSizes, @PreviewFontScales, @PreviewLightDark, and @PreviewDynamicColors, so that with one single annotation, you can preview your Compose UI in common scenarios.

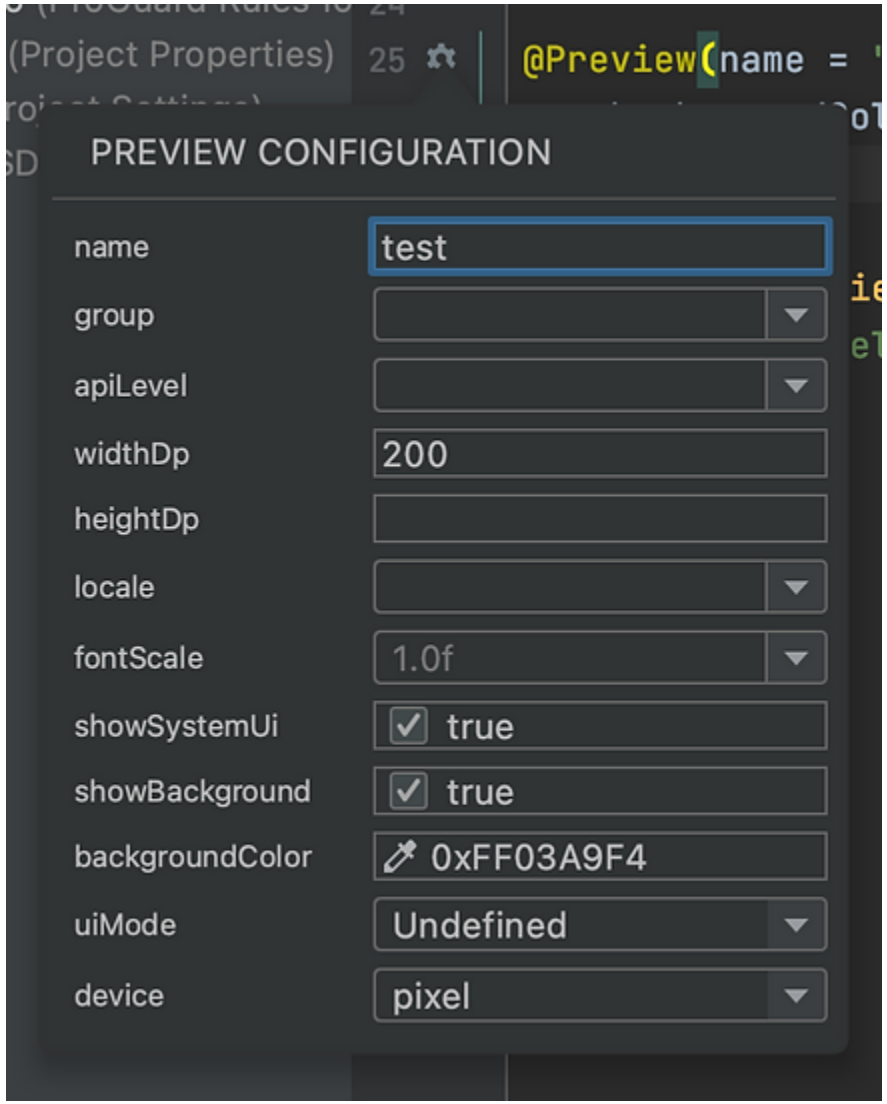



**More:**

https://developer.android.com/develop/ui/compose/tooling/previews

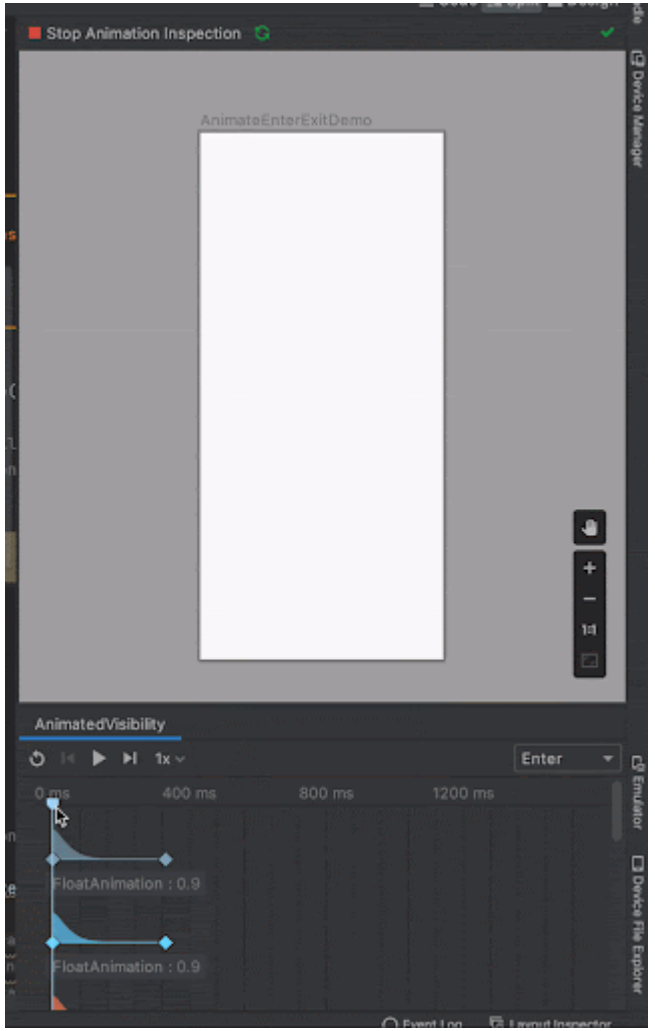
# Adjust preview settings

```
@Preview( showBackground = true, showSystemUi = true)
@Composable
fun DefaultPreview() {
    Row {  this: RowScope
        TextContainer()
    }
}
```



## Preview animation

If an animation is described in a composable preview,
You can inspect the exact value of each animated value at a given time, pause the animation, loop it, fast-forward it, or slow it down to help you debug the animation throughout its transitions:




Animation Preview automatically detects inspectable animations, which are indicated by the Start Animation Preview icon and the Run icon.

**More:**

https://developer.android.com/develop/ui/compose/tooling/animation-preview

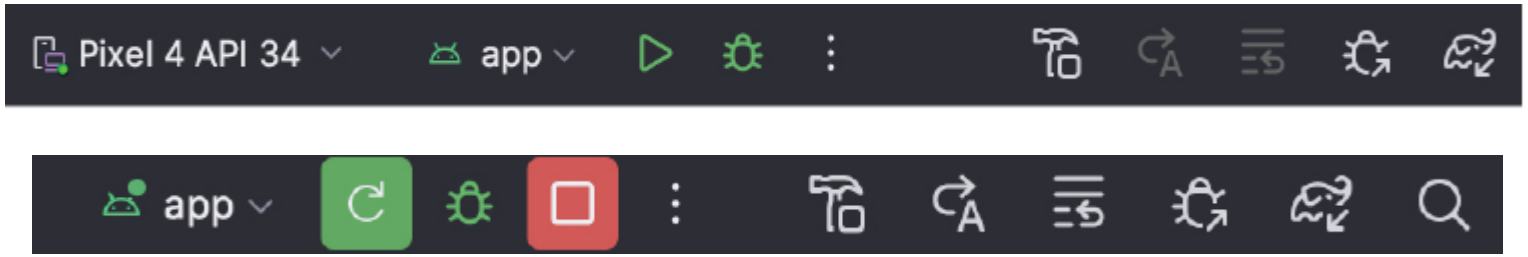
# Run Android Compose Application

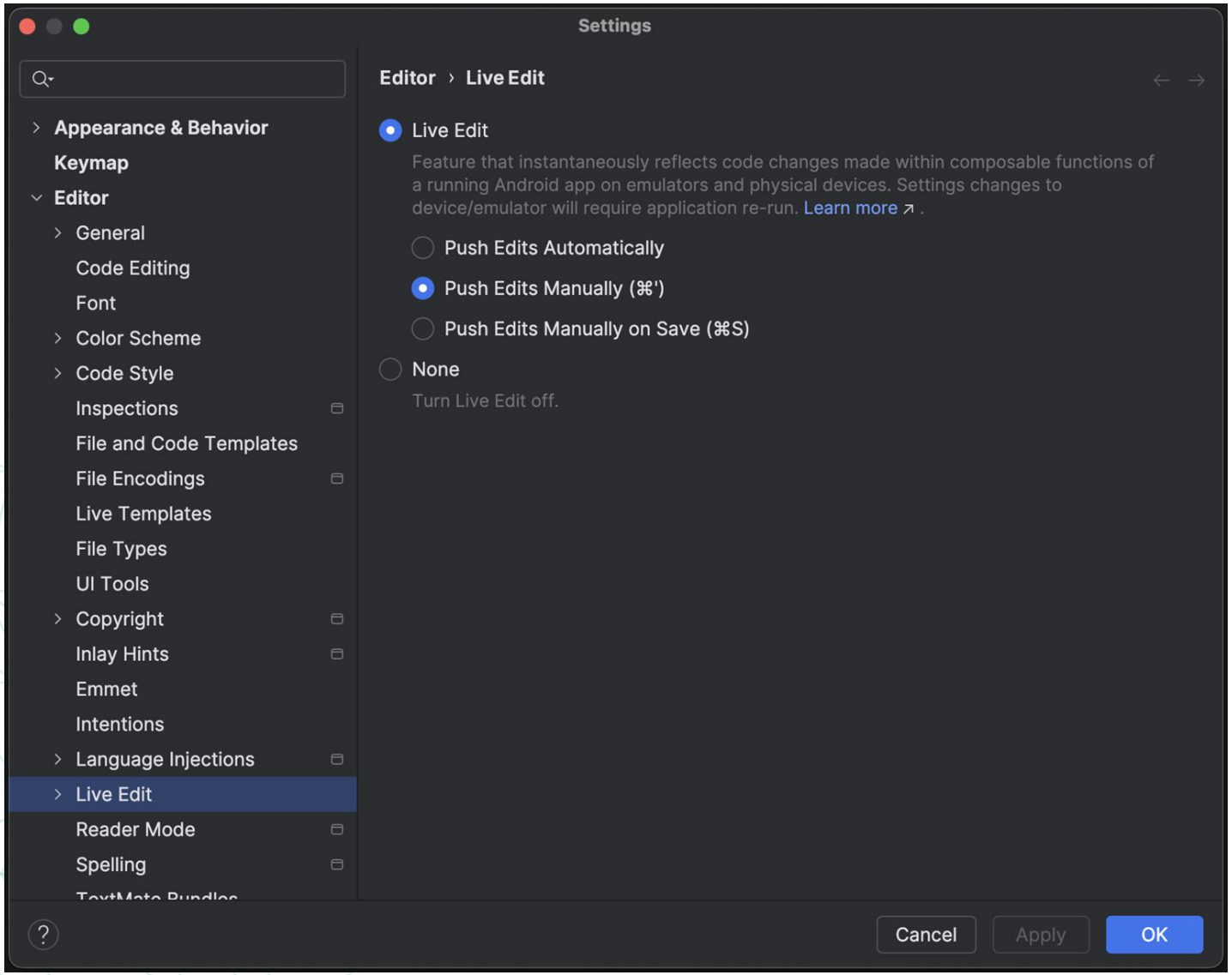
1- Starting the Emulator

2- Running the Application in the AVD

3- Real-time updates with Live Edit

4- Stopping a Running Application




Settings
Editor › Live Edit
Appearance & Behavior
Keymap
Editor
General
Code Editing
Font
Color Scheme
Code Style
Inspections
File and Code Templates
File Encodings
Live Templates
File Types
UI Tools
Copyright
Inlay Hints
Emmet
Intentions
Language Injections
Live Edit
Reader Mode
Spelling
Live Edit
Feature that instantaneously reflects code changes made within composable functions of a running Android app on emulators and physical devices. Settings changes to device/emulator will require application re-run. Learn more ↗ .
Push Edits Automatically
Push Edits Manually (⌘')
Push Edits Manually on Save (⌘S)
None
Turn Live Edit off.
Cancel
Apply
OK

# Stateful vs. stateless composables

**State**, in the context of Compose, is defined as being **any value that can change during the execution of an app**. For example, a slider position value, the string entered into a text, or the current setting of a check box are all forms of state.

Composable functions can be categorized as **stateless** or **stateful**, depending on whether they hold and manage the state internally.

## Stateful

- **Holds and manages state internally** using Compose's state management APIs like **remember** or **rememberSaveable.**
- Reacts to state changes and updates the UI accordingly.
- Less reusable.

```
@Composable
fun Counter() {
    var count by remember { mutableStateOf(0) } // Internal state
    Button(onClick = { count++ }) {
        Text("Count: $count")
    }
}
```

## Stateless

- **Does not manage its own state**.
- Hoists its state (callers pass the state to it)
- More reusable.
  It produces the same output (UI) for the same input every time.

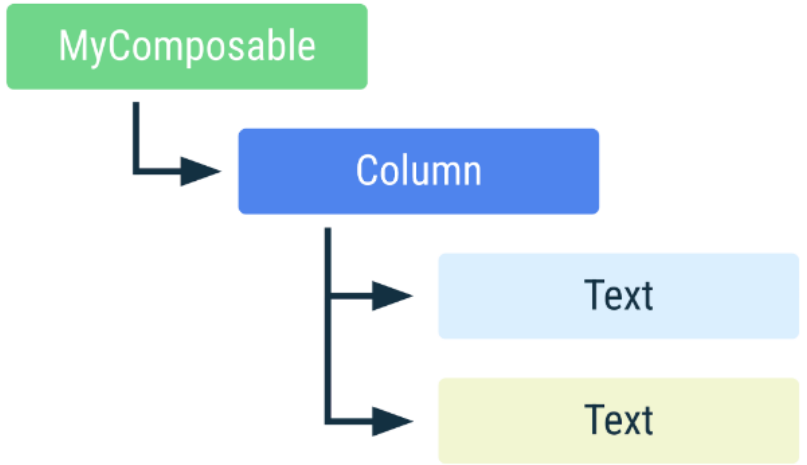
```
@Composable
fun Greeting(name: String) {
    Text(text = "Hello, $name!")
}
```

# Composable hierarchy

```
@Composable
fun MyComposable() {
    Column {
        Text("Hello")
        Text("World")
    }
}
```

Composable functions can call other composables,
essentially building a hierarchical tree of user interface components.

Consider, for example, the following composable hierarchy tree
for an Android app where each element in the tree
(except the ComponentActivity) is a composable function




The composables bundled with Compose fall into three categories,
referred to as

- Layout,
- Foundation,
- and Material Design components.

**Layout components** provide a way to define both **how** components are **positioned** on the screen and **how** those components **behave** in relation to each other.

**The following are all layout composables :**

- Box
- BoxWithConstraints
- Column
- ConstraintLayout
- Row
- Spacer

**Foundation components** are a set of minimal components that provide basic user interface functionality.
While these components do not, by default, impose a specific style or theme,
they can be customized to provide any look and behavior you need for your app.

**The following lists the set of foundation components:**

- BaseTextField
- Canvas
- Image
- Icon
- LazyColumn
- LazyRow
- BorderStroke, Shape, Background
- Text

The **Material Design components**, on the other hand, have been designed so that they **match Google's Material theme guidelines** and include the **following composables**:

- AlertDialog
- Button
- FloatingActionButton
- Card
- CircularProgressIndicator
- DropdownMenu
- Checkbox
- LinearProgressIndicator
- ModalDrawer
- RadioButton
- Scaffold
- Surface
- TopAppBar, CenterAlignedTopAppBar (headers)
- NavigationBar, NavigationRail, NavigationDrawer
- BottomNavigation
- Slider
- Snackbar
- Switch
- TextField

# Basic layouts in Compose

Jetpack Compose makes it much easier to design and build your app's UI.

Compose **transforms state into UI elements**, via:

1. Composition of elements
2. Layout of elements
3. Drawing of elements


**Composable functions** are the **basic building blocks of Compose**.

A composable function is a function **emitting** Unit that describes some part of your **UI**.

The function takes some input and generates what's shown on the screen.

A composable function might emit several UI elements. However, if you don't provide guidance on how they should be arranged, Compose might arrange the elements in a way you don't like.

# Standard Layout Components

## Column

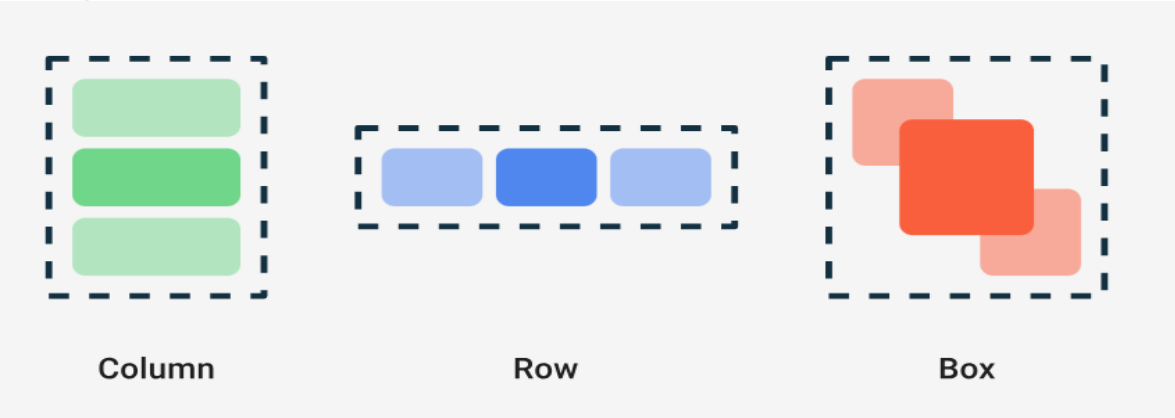
Use **Column** to place items **vertically** on the screen.

## Row

use **Row** to place items **horizontally** on the screen.

## Box

Use **Box** to put elements **on top of each other**.



## Example

The UI tree is laid out in a single pass. Each node is first asked to measure itself, then measure any children recursively, passing size constraints down the tree to children. Then, leaf nodes are sized and placed, with the resolved sizes and placement instructions passed back up the tree.

In the SearchResult example, the UI tree layout follows this order:

1. The root node Row is asked to measure.
2. The root node Row asks its first child, Image, to measure.
3. Image is a leaf node (that is, it has no children), so it reports a size and returns placement instructions.

```
@Composable
fun SearchResult() {
    Row {
        Image(
            // ...
        )
        Column {
            Text(
                // ...
            )
            Text(
                // ...
            )
        }
    }
}
```

## Example

4. The root node Row asks its second child, Column, to measure.
5. The Column node asks its first Text child to measure.
6. The first Text node is a leaf node, so it reports a size and returns placement instructions.
7. The Column node asks its second Text child to measure.
8. The second Text node is a leaf node, so it reports a size and returns placement instructions.
9. Now that the Column node has measured, sized, and, placed its children, it can determine its own size and placement.
10. Now that the root node Row has measured, sized, and placed its children, it can determine its own size and placement.

```
@Composable
fun SearchResult() {
    Row {
        Image(
            // ...
        )
        Column {
            Text(
                // ...
            )
            Text(
                // ...
            )
        }
    }
}
```

## Layout Alignment

Row **vertical** alignment parameter

| | |
|---|---|
| Alignment.Top | Aligns the content at the **top of** the Row content area. |
| Alignment.CenterVertically | Positions the content in the **vertical center of the Row** content area. |
| Alignment.Bottom | Aligns the content at the **bottom of** the Row content area. |

Column **horizontal** alignment parameter

| | |
|---|---|
| Alignment.Start | Aligns the content at the horizontal **start of** the Column content area. |
| Alignment.CenterHorizontally | Positions the content in the **horizontal center of the Column** content area |
| Alignment.End | Aligns the content at the horizontal **end of** the Column content area. |

## Layout arrangement positioning

Arrangement controls child positioning along the same axis as the container (i.e., horizontally for Rows and vertically for Columns).

### Horizontal Arrangement for Row

| | |
|---|---|
| Arrangement.Start | Aligns the content at the **horizontal start of the Row** content area. |
| Arrangement.Center | Positions the content in the **horizontal center of the Row** content area. |
| Arrangement.End | Aligns the content at the **horizontal end of the Row** content area**.** |

### Vertical Arrangement for Column

| | |
|---|---|
| Arrangement.Top | Aligns the content at the **top of the Column** content area. |
| Arrangement.Center | Positions the content in the **vertical center of the Column** content area. |
| Arrangement.Bottom | Aligns the content at the **bottom of the Column** content area. |

## Layout arrangement spacing

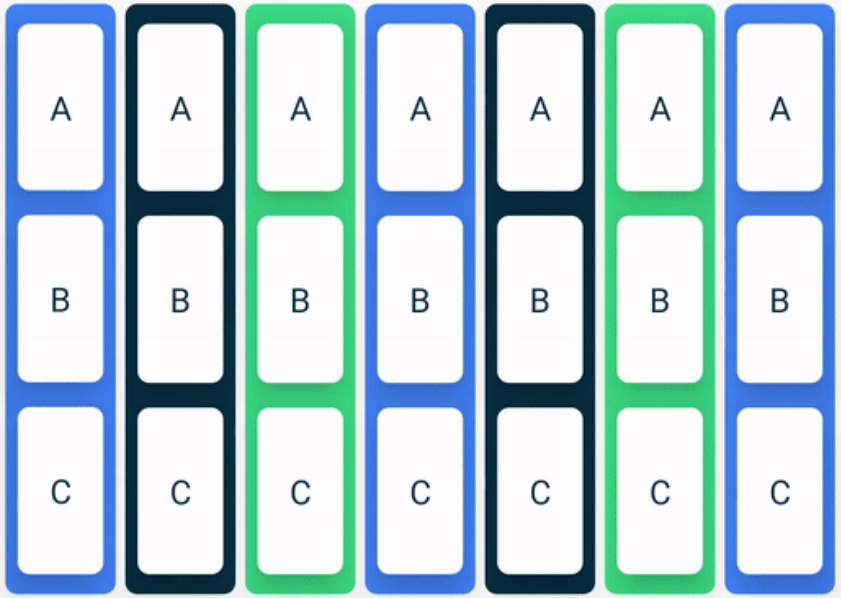
Arrangement spacing controls how the child components in a Row or Column are spaced across the content area

| | |
|---|---|
| SpaceEvenly | Arrangement places child elements across the main axis, **including free space before the first and after the last child** |
| SpaceBetween | Arrangement places child elements across the main axis **without free space before the first and after the last child**. |
| SpaceAround | Arrangement places child elements across the main axis **with half of the free space before the first and after the last child.** |

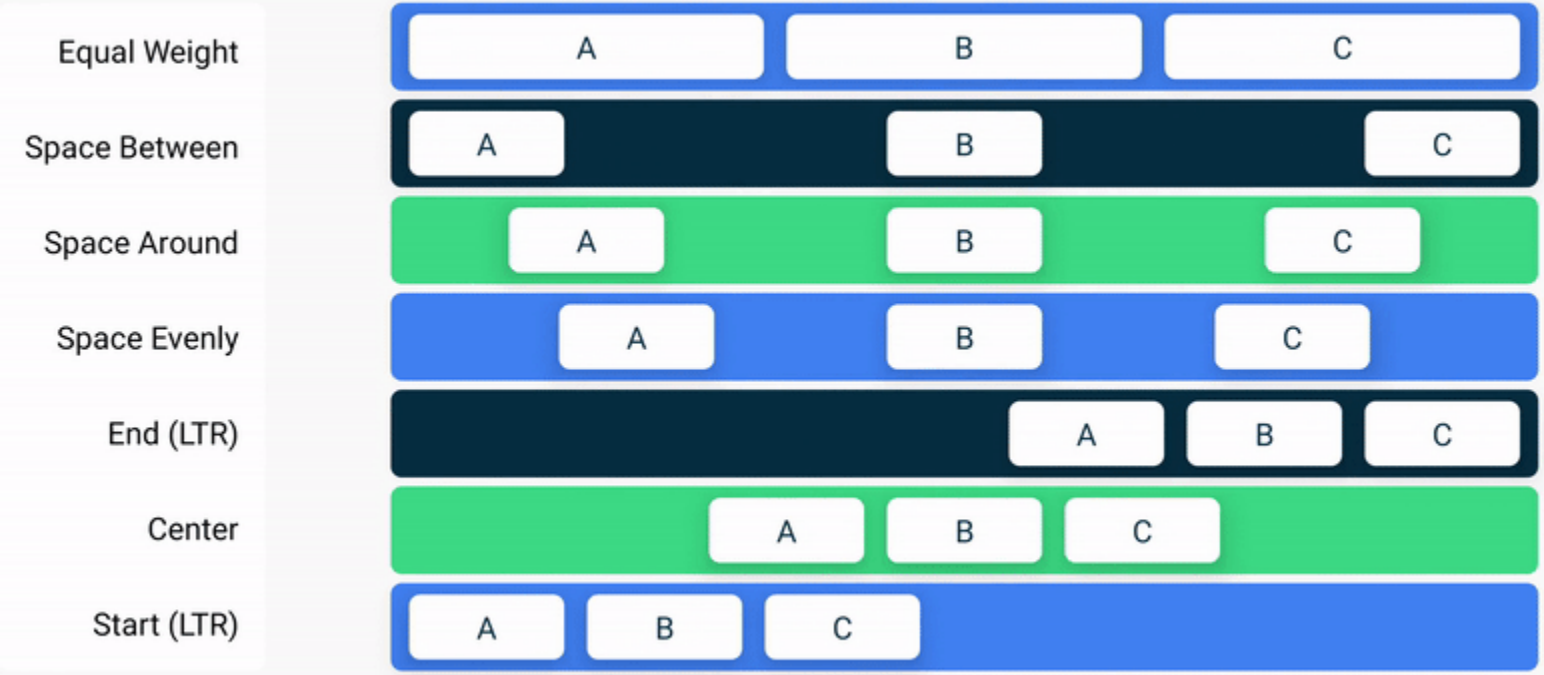
رواد مصر الرقمية

# Column

Equal Weight | Space Between | Space Around | Space Evenly | Top | Center | Bottom




# Row



# Row and Column scope modifiers

The children of a Row or Column are said to be within the scope of the parent.

These two scopes (**RowScope** and **ColumnScope**) provide a set of additional modifier functions that can be applied to change the behavior and the appearance of individual children within a Row or Column

**ColumnScope** includes the following modifiers for controlling the position of child components:

| | |
|---|---|
| Modifier.align() | Allows the child to be aligned **horizontally** using Alignment.CenterHorizontally, Alignment.Start, and Alignment.End values. |
| Modifier.alignBy() | Aligns a child **horizontally** with other siblings on which the alignBy() modifier has also been applied. |
| Modifier.weight() | Sets the **height** of the child **relative to** the **weight values** assigned to its siblings. |

**RowScope** provides the following additional modifier functions to Row children

| | |
|---|---|
| Modifier.align() | Allows the child to be aligned **vertically** using Alignment.CenterVertically, Alignment.Top, and Alignment.Bottom values. |
| Modifier.alignBy() | Aligns a child with other siblings on which the alignBy() modifier has also been applied. Alignment may be performed by **baseline** or using **custom alignment line configurations**. |
| Modifier.alignByBaseline() | Aligns the baseline of a child with anysiblings that have also been configured by either the alignBy() or alignByBaseline() modifier |
| Modifier.paddingFrom() | Allows padding to be added to the alignment line of a child. |
| Modifier.weight() | Sets the **width** of the child **relative to** the **weight values** assigned to its siblings. |

## Scope modifier weights

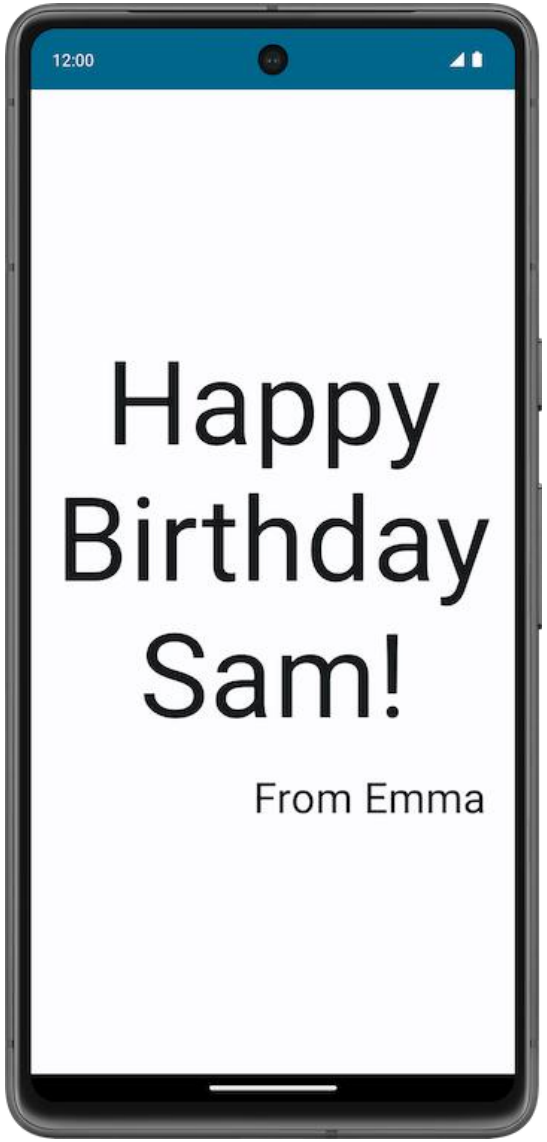
The RowScope weight modifier allows the width of each child to be specified relative to its siblings.

It works by assigning each child a weight percentage (between 0.0 and 1.0)

Two children assigned a weight of 0.5, for example, would each occupy half of the available space.

## Exercise 1

An Android app that displays a birthday greeting in text format, which looks like this screenshot

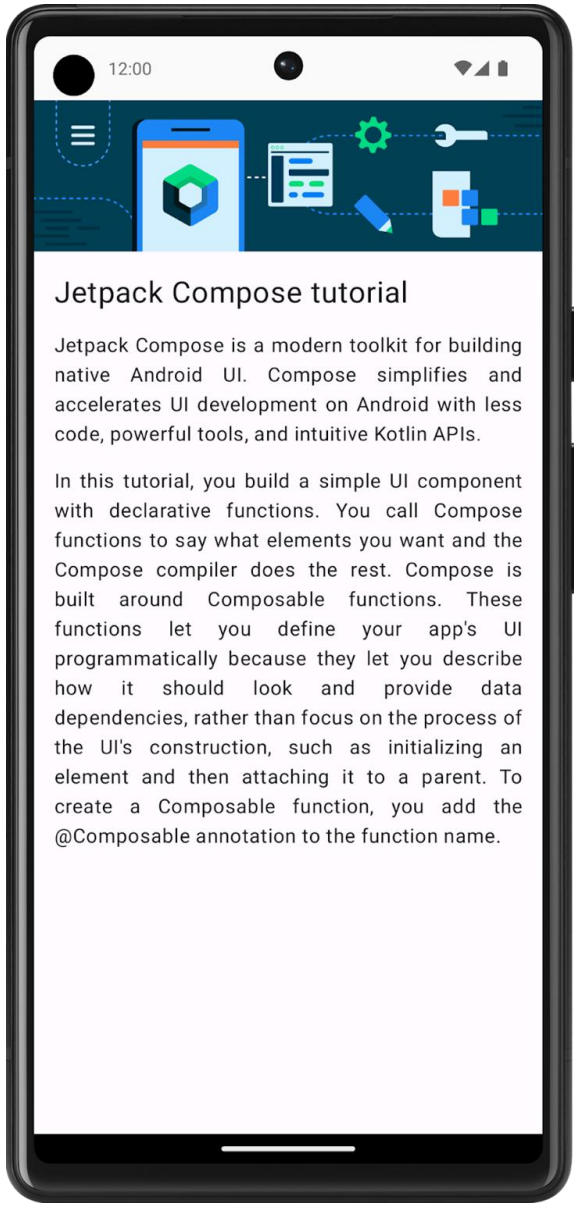



https://developer.android.com/codelabs/basic-android-kotlin-compose-text-composables#0

# Exercise 2

An Android app that displays a Compose article

https://developer.android.com/codelabs/basic-android-kotlin-compose-composables-practice-problems#0

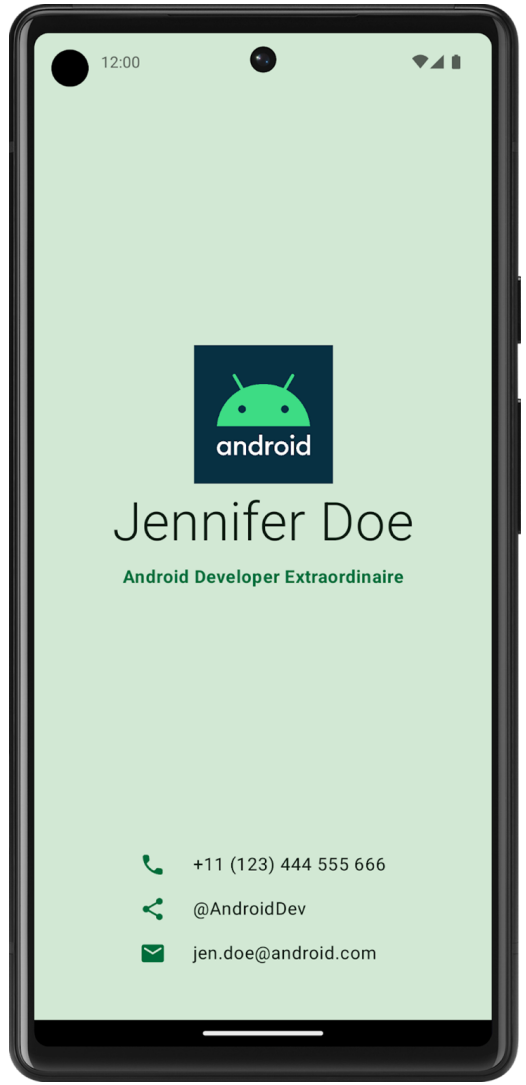


# Assignment 1

Create a Business Card app

Follow the instructions:

https://developer.android.com/codelabs/basic-android-kotlin-compose-business-card#0



# Quiz Time

# Key Takeaways

- Jetpack Compose is a modern toolkit for building Android UI.
  Jetpack Compose simplifies and accelerates UI development on Android with less code, powerful tools, and intuitive Kotlin APIs.

- The user interface (UI) of an app is what you see on the screen: text, images, buttons, and many other types of elements.

- Composable functions are the basic building block of Compose.
  A composable function is a function that describes some part of your UI.

- The Composable function is annotated with the `@Composable` annotation.
  This annotation informs the Compose compiler that this function is intended to convert data into UI.

- The three basic standard layout elements in Compose are `Column`, `Row`, and `Box`.
  They are Composable functions that take Composable content, so you can place items inside.
  For example, each child within a `Row` will be placed horizontally next to each other.

# THANK YOU